

Кафедра комп'ютерних
систем і мереж

ЛАБОРАТОРНА РОБОТА №14

з дисципліни **Теорія алгоритмів і структури даних**

Тема:

Способи обходу графів: DFS та BFS алгоритми

Виконала:

ст. гр. КІ-25-1

Лаврів А.

Перевірив:

Гарасимів Т.Г.

м. Івано-Франківськ

2026

Мета: Набуття практичних вмінь і навичок при представленні заданих графів різними способами та можливістю їх комп'ютерної реалізації.

Завдання:

Для неорієнтованих та орієнтованих графів зі своїх варіантів (варіант вибирається згідно номера по журналу, варіанти завдань наведено у додатку Б) виконати наступні завдання:

1. Написати програму, яка виконуватиме наступні завдання:

- за заданою матрицею суміжності побудувати матрицю інцидентності;
- за заданою матрицею інцидентності побудувати список ребер;
- за заданою матрицею суміжності побудувати список суміжності.
- за заданою матрицею інцидентності побудувати матрицю суміжності;
- за заданою матрицею суміжності побудувати список ребер;
- за заданою матрицею інцидентності побудувати список суміжності.

Код:

```
#include <iostream>
#include <string>
#include <vector>
#include <queue>
#include <stack>
#include <iomanip>
using namespace std;
```

```
class Graph {
    int n;
    bool directed;
    vector<string> labels;
    vector<vector<int>> adjMatrix;

    int idx(const string& name) const {
        for (int i = 0; i < n; i++)
            if (labels[i] == name) return i;
        return -1;
    }
}
```

public:

```

Graph(const vector<string>& vertexLabels, bool isDirected = false)
: n(vertexLabels.size()), directed(isDirected),
  labels(vertexLabels),
  adjMatrix(vertexLabels.size(), vector<int>(vertexLabels.size(), 0))
{}

```

```

void addEdge(const string& from, const string& to) {
    int u = idx(from), v = idx(to);
    if (u < 0 || v < 0) { cout << "Невірна вершина!" << endl; return; }
    adjMatrix[u][v]++;
    if (!directed) adjMatrix[v][u]++;
}

```

```

void addLoop(const string& v) {
    int i = idx(v);
    if (i < 0) return;
    adjMatrix[i][i]++;
}

```

```

void printAdjMatrix() const {
    cout << "Матриця суміжності:" << endl;
    cout << "    ";
    for (int i = 0; i < n; i++)
        cout << setw(4) << labels[i];
    cout << endl;
    cout << "    " << string(4 * n, '-') << endl;
    for (int i = 0; i < n; i++) {
        cout << setw(3) << labels[i] << " |";
        for (int j = 0; j < n; j++)
            cout << setw(4) << adjMatrix[i][j];
        cout << endl;
    }
}

```

```

void printAdjList() const {
    cout << "Списки суміжності:" << endl;

```

```

for (int i = 0; i < n; i++) {
    cout << " " << labels[i] << " -> { ";
    bool first = true;
    for (int j = 0; j < n; j++) {
        int cnt = adjMatrix[i][j];
        if (i == j && cnt > 0) {

            for (int k = 0; k < cnt; k++) {
                if (!first) cout << ", ";
                cout << labels[j] << "(петля)";
                first = false;
            }
        } else {
            for (int k = 0; k < cnt; k++) {
                if (!first) cout << ", ";
                cout << labels[j];
                first = false;
            }
        }
    }
    cout << " }" << endl;
}
}

```

```

void DFS(const string& startLabel) const {
    int start = idx(startLabel);
    if (start < 0) { cout << "Невірна стартова вершина!" << endl; return; }

    vector<int>    dfsNum(n, 0);
    vector<bool>    visited(n, false);
    vector<bool>    edgeMarked(n * n, false);

    vector<int>    parent(n, -1);
    vector<string>    edgeType(n * n, "");

    stack<int>    stk;
    int          counter = 1;

```

```
cout << " +-----+-----+-----+" << endl;
cout << " | Вершина | DFS № | Вміст стеку |" << endl;
cout << " +-----+-----+-----+" << endl;
```

```
stk.push(start);
visited[start] = true;
dfsNum[start] = counter++;
```

```
auto stackStr = [&]() {
    stack<int> tmp = stk;
    vector<int> elems;
    while (!tmp.empty()) { elems.push_back(tmp.top()); tmp.pop(); }
    string s = "[ ";
    for (int i = elems.size() - 1; i >= 0; i--) {
        s += labels[elems[i]];
        if (i > 0) s += ", ";
    }
    s += " ]";
    return s;
};
```

```
cout << " | " << setw(9) << left << labels[start]
    << " | " << setw(6) << dfsNum[start]
    << " | " << setw(25) << left << stackStr() << " | " << right << endl;
```

```
vector<string> traversal = {labels[start]};
```

```
while (!stk.empty()) {
    int u = stk.top();
```

```
bool found = false;
for (int v = 0; v < n; v++) {
    if (adjMatrix[u][v] == 0) continue;
    if (u == v) continue;
```

```

int edgeKey = u * n + v;
if (!edgeMarked[edgeKey]) {
    edgeMarked[edgeKey] = true;
    if (!directed) edgeMarked[v * n + u] = true;

    if (!visited[v]) {

        edgeType[edgeKey] = "дерева";
        parent[v] = u;
        visited[v] = true;
        dfsNum[v] = counter++;
        stk.push(v);
        traversal.push_back(labels[v]);

        cout << " | " << setw(9) << left << labels[v]
            << " | " << setw(6) << dfsNum[v]
            << " | " << setw(25) << left << stackStr() << " | " << right << endl;
        found = true;
        break;
    } else {

        edgeType[edgeKey] = directed ? "зворотне/поперечне" : "зворотне";
    }
}
}

if (!found) {
    stk.pop();
    if (!stk.empty()) {
        cout << " | " << setw(9) << left << ("вихід " + labels[u])
            << " | " << setw(6) << "-"
            << " | " << setw(25) << left << stackStr() << " | " << right << endl;
    }
}

cout << " +-----+-----+-----+-----+" << endl;

```

```

cout << "\n Порядок обходу DFS: ";
for (int i = 0; i < (int)traversal.size(); i++) {
    cout << traversal[i];
    if (i + 1 < (int)traversal.size()) cout << " -> ";
}
cout << endl;

```

```

cout << "\n DFS-номери вершин:" << endl;
for (int i = 0; i < n; i++)
    cout << "    DFS(" << labels[i] << ") = " << dfsNum[i] << endl;

```

```

cout << "\n Ребра каркасного дерева : " << endl;
for (int v = 0; v < n; v++) {
    if (parent[v] >= 0)
        cout << "    " << labels[parent[v]] << " --> " << labels[v] << endl;
}
}

```

```

void BFS(const string& startLabel) const {
    int start = idx(startLabel);
    if (start < 0) { cout << "Невірна стартова вершина!" << endl; return; }

```

```

vector<int> bfsNum(n, 0);
vector<bool> visited(n, false);
vector<int> parent(n, -1);

```

```

queue<int> q;
int counter = 1;

```

```

auto queueStr = [&]() {
    queue<int> tmp = q;
    string s = "[ ";
    bool first = true;
    while (!tmp.empty()) {
        if (!first) s += ", ";
        s += labels[tmp.front()];
    }
}

```

```

        tmp.pop();
        first = false;
    }
    s += " ]";
    return s;
};

```

```

cout << " +-----+-----+-----+" << endl;
cout << " | Вершина | BFS № | Вміст черги |" << endl;
cout << " +-----+-----+-----+" << endl;

```

```

visited[start] = true;
bfsNum[start] = counter++;
q.push(start);

```

```

cout << " | " << setw(9) << left << labels[start]
    << " | " << setw(6) << bfsNum[start]
    << " | " << setw(25) << left << queueStr() << " |" << right << endl;

```

```

vector<string> traversal = {labels[start]};

```

```

while (!q.empty()) {
    int u = q.front();
    q.pop();

```

```

    for (int v = 0; v < n; v++) {
        if (adjMatrix[u][v] == 0) continue;
        if (u == v) continue;

```

```

        if (!visited[v]) {
            visited[v] = true;
            bfsNum[v] = counter++;
            parent[v] = u;
            q.push(v);
            traversal.push_back(labels[v]);

```

```

        cout << " | " << setw(9) << left << labels[v]
            << " | " << setw(6) << bfsNum[v]

```

```

        << " | " << setw(25) << left << queueStr() << " |" << right << endl;
    }
}
}

cout << " +-----+-----+-----+-----+" << endl;

cout << "\n Порядок обходу BFS: ";
for (int i = 0; i < (int)traversal.size(); i++) {
    cout << traversal[i];
    if (i + 1 < (int)traversal.size()) cout << " -> ";
}
cout << endl;

cout << "\n BFS-номери вершин:" << endl;
for (int i = 0; i < n; i++)
    cout << "   BFS(" << labels[i] << ") = " << bfsNum[i] << endl;

cout << "\n Ребра каркасного дерева (широтного):" << endl;
for (int v = 0; v < n; v++) {
    if (parent[v] >= 0)
        cout << "   " << labels[parent[v]] << " --> " << labels[v] << endl;
}
}
};

```

```

int main() {

```

```

    cout << " НЕОРИЄНТОВАНИЙ ГРАФ " << endl;

```

```

    Graph undirected({"a","b","c","d","e","f"}, false);

```

```
undirected.addLoop("a");
undirected.addLoop("b");
```

```
undirected.addEdge("a", "b");
undirected.addEdge("b", "c");
undirected.addEdge("b", "d");
undirected.addEdge("c", "d");
undirected.addEdge("c", "d");
undirected.addEdge("c", "e");
undirected.addEdge("d", "f");
undirected.addEdge("e", "f");
```

```
cout << endl;
undirected.printAdjMatrix();
cout << endl;
undirected.printAdjList();
```

```
cout << "\n--- DFS-обхід, початок з вершини 'a' ---" << endl;
undirected.DFS("a");
```

```
cout << "\n--- BFS-обхід, початок з вершини 'a' ---" << endl;
undirected.BFS("a");
```

```
cout << " ОРІЄНТОВАНИЙ ГРАФ (варіант 11)" << endl;
```

```
Graph directed({"a","b","c","d","e","f"}, true);
```

```
directed.addEdge("a", "b");
directed.addEdge("a", "c");
directed.addEdge("b", "c");
directed.addEdge("b", "d");
directed.addEdge("c", "e");
directed.addEdge("c", "f");
```

```
directed.addEdge("d", "a");
directed.addEdge("d", "e");
directed.addEdge("e", "f");
```

```
cout << endl;
directed.printAdjMatrix();
cout << endl;
directed.printAdjList();
```

```
cout << "\n--- DFS-обхід орграфу, початок з вершини 'a' ---" << endl;
directed.DFS("a");
```

```
cout << "\n--- BFS-обхід орграфу, початок з вершини 'a' ---" << endl;
directed.BFS("a");
```

```
return 0;
}

\
```

```

НЕОРІЄНТОВАНИЙ ГРАФ

Матриця суміжності:
      a  b  c  d  e  f
-----
a |  1  1  0  0  0  0
b |  1  1  1  1  0  0
c |  0  1  0  2  1  0
d |  0  1  2  0  0  1
e |  0  0  1  0  0  1
f |  0  0  0  1  1  0

Списки суміжності:
a -> { a(петля), b }
b -> { a, b(петля), c, d }
c -> { b, d, d, e }
d -> { b, c, c, f }
e -> { c, f }
f -> { d, e }

--- DFS-обхід, початок з вершини 'a' ---
+-----+-----+-----+
| Вершина | DFS № | Вміст стеку |
+-----+-----+-----+
| a       | 1     | [ a ]       |
| b       | 2     | [ a, b ]    |
| c       | 3     | [ a, b, c ] |
| d       | 4     | [ a, b, c, d ] |
| f       | 5     | [ a, b, c, d, f ] |
| e       | 6     | [ a, b, c, d, f, e ] |
| вихід e | -     | [ a, b, c, d, f ] |
| вихід f | -     | [ a, b, c, d ] |
| вихід d | -     | [ a, b, c ] |
| вихід c | -     | [ a, b ] |
| вихід b | -     | [ a ] |
+-----+-----+-----+

Порядок обходу DFS: a -> b -> c -> d -> f -> e

```

Рис. 1 - Демонстрація роботи коду

```

DFS-номери вершин:
DFS(a) = 1
DFS(b) = 2
DFS(c) = 3
DFS(d) = 4
DFS(e) = 6
DFS(f) = 5

Ребра каркасного дерева :
a --> b
b --> c
c --> d
f --> e
d --> f

--- BFS-обхід, початок з вершини 'a' ---
+-----+-----+-----+
| Вершина | BFS № | Вміст черги |
+-----+-----+-----+
| a       | 1     | [ a ]       |
| b       | 2     | [ b ]       |
| c       | 3     | [ c ]       |
| d       | 4     | [ c, d ]    |
| e       | 5     | [ d, e ]    |
| f       | 6     | [ e, f ]    |
+-----+-----+-----+

Порядок обходу BFS: a -> b -> c -> d -> e -> f

BFS-номери вершин:
BFS(a) = 1
BFS(b) = 2
BFS(c) = 3
BFS(d) = 4
BFS(e) = 5
BFS(f) = 6

```

Рис.2 - Демонстрація роботи коду

```

Ребра каркасного дерева (широтного):
a --> b
b --> c
b --> d
c --> e
d --> f

ОРІЄНТОВАНИЙ ГРАФ (варіант 11)

Матриця суміжності:
      a  b  c  d  e  f
-----
a |  0  1  1  0  0  0
b |  0  0  1  1  0  0
c |  0  0  0  0  1  1
d |  1  0  0  0  1  0
e |  0  0  0  0  0  1
f |  0  0  0  0  0  0

Списки суміжності:
a -> { b, c }
b -> { c, d }
c -> { e, f }
d -> { a, e }
e -> { f }
f -> { }

```

Рис. 3 - Демонстрація роботи коду

```

--- DFS-обхід орграфу, початок з вершини 'a' ---
+-----+-----+-----+
| Вершина | DFS № | Вміст стеку |
+-----+-----+-----+
| a       | 1     | [ a ]       |
| b       | 2     | [ a, b ]    |
| c       | 3     | [ a, b, c ] |
| e       | 4     | [ a, b, c, e ] |
| f       | 5     | [ a, b, c, e, f ] |
| вихід f | -     | [ a, b, c, e ] |
| вихід e | -     | [ a, b, c ]  |
| вихід c | -     | [ a, b ]     |
| d       | 6     | [ a, b, d ]  |
| вихід d | -     | [ a, b ]     |
| вихід b | -     | [ a ]        |
+-----+-----+-----+

Порядок обходу DFS: a -> b -> c -> e -> f -> d

DFS-номери вершин:
DFS(a) = 1
DFS(b) = 2
DFS(c) = 3
DFS(d) = 6
DFS(e) = 4
DFS(f) = 5

Ребра каркасного дерева :
a --> b
b --> c
b --> d
c --> e
e --> f

```

Рис. 4 - Демонстрація роботи коду

```

--- BFS-обхід орграфу, початок з вершини 'a' ---
+-----+-----+-----+
| Вершина | BFS № | Вміст черги |
+-----+-----+-----+
| a       | 1     | [ a ]       |
| b       | 2     | [ b ]       |
| c       | 3     | [ b, c ]    |
| d       | 4     | [ c, d ]    |
| e       | 5     | [ d, e ]    |
| f       | 6     | [ d, e, f ] |
+-----+-----+-----+

Порядок обходу BFS: a -> b -> c -> d -> e -> f

BFS-номери вершин:
BFS(a) = 1
BFS(b) = 2
BFS(c) = 3
BFS(d) = 4
BFS(e) = 5
BFS(f) = 6

Ребра каркасного дерева (широтного):
a --> b
a --> c
b --> d
c --> e
c --> f

```

Рис. 5 - Демонстрація роботи коду

Висновок: У ході виконання лабораторної роботи було досліджено алгоритми обходу графів у глибину (DFS) та у ширину (BFS). Було реалізовано програми обходу графів та проаналізовано особливості роботи кожного алгоритму. Отримано практичні навички роботи з графами та алгоритмами їх обходу мовою C++.